

第3章 流程控制

建议学时:2-4 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 if 条件的"真值"规则:JS 不要求布尔,6 个假值之外都是真
- 掌握 switch 与 C 的差异:严格 ===、无默认 fallthrough、共享 case
- 熟练使用四种循环:for、while、do-while、for...of,并分清 for...in 的用途
- 掌握带标签的 break/continue,理解它是 C 的 goto 的安全替代
- 彻底理解循环里 var 的经典闭包 bug 与 let 的块作用域修复
- 掌握数组遍历的现代写法:for...of、forEach、map/filter 的选择
- 理解"C 中怎么做→JS 中怎么做":循环变量作用域与迭代方式

💡 从 C 到 JavaScript:本章主题如何迁移

流程控制是两种语言最接近的部分:if/else、for、while、do-while、switch、break、continue 一应俱全,大括号风格都一致。但也藏着三处暗礁。

第一处暗礁是 **条件不要求是布尔**:C 里 `if (p)` 检查指针非空,JS 里 `if (x)` 检查"真值"——但空字符串、0、NaN、null、undefined 都是假,这在 C 里只有 0 是假。写 `if (name)` 判断"名字非空"在生产里是常见 bug 源:'0'、'false' 都是真值,空串才触发分支。

第二处暗礁是 **switch 的语义**:C 的 switch 只支持整型并默认 fallthrough;JS 的 switch 用 === 比较,支持任意类型,且自动穿透(每个 case 结尾的 break 是惯例而非必需)。但 JS 保留了"共享 case"能力: `case 1: case 2:` 叠写仍合法。

第三处暗礁是 **循环与闭包**:C 的 `for (int i = 0; i < 3; i++)` 里 i 是块变量;JS 的 var 是函数作用域,循环结束时 i 只剩下最终值,回调里读 i 全部读到同一个值——这就是闻名天下的"循环里 var 的闭包 bug"。现代 JS 用 let 声明循环变量,彻底解决。

JS 还提供了 C 没有的迭代器遍历:for...of 按顺序取每个元素,不需要下标;数组还有 map/filter/reduce 等函数式遍历。读数组代码时,先判断它用哪种风格,再逐段理解。

📦 前置知识自查

- C 的 if/else 与悬挂 else(else 与最近的 if 结合)
- C 的 switch 语义:整型、fallthrough、break
- C 的 for/while/do-while 与 break/continue
- C 的 goto 与标签
- C 的循环变量作用域(块作用域)

本章学习路线

1. **第一阶段(if 与 switch):**读 §3.1–§3.2,重点记真值表与 switch 的 === 语义
2. **第二阶段(循环):**读 §3.3–§3.4,把每个循环例子在 REPL 跑一遍
3. **第三阶段(闭包 bug 与数组遍历):**读 §3.5–§3.7,亲手复现 var 闭包 bug 并修复
4. **第四阶段(实验):**完成章末"文本菜单 + 素数筛选 + 统计"
5. **验收标准:**能独立写出循环里用 let 的正确回调代码,并解释 var 为什么错

本章知识导图

- 第3章 流程控制
 - §3.1 if/else 与真值判断、悬挂 else
 - §3.2 switch:严格 ===、共享 case、与 C 对照
 - §3.3 循环:for / while / do-while / for...of / for...in
 - §3.4 break/continue 与标签
 - §3.5 goto 的替代:提前返回与标签 break
 - §3.6 循环变量捕获:var 闭包 bug 与 let 修复
 - §3.7 数组遍历三风格与现代写法、无限循环对照

§3.1 if / else:条件按真值判断

```
const score = 85;
if (score >= 90) {
  console.log('优秀');
} else if (score >= 60) {
  console.log('及格');
} else {
  console.log('不及格');
}

// 真值判断:JS 不要求条件为布尔
const name = '';
if (name) { // '' 是假值,不进分支
  console.log('有名字');
} else {
  console.log('名字为空');
}

const count = 0;
if (count) { // 0 是假值!这里常常是 bug
  console.log('有计数');
} else {
  console.log('计数为 0'); // 走到这里
}

// 生产建议:明确意图
if (count > 0) { // 明确:大于 0
  console.log('有计数');
}
```

⚠ 常见误区 3.1:if (arr.length) 与 if (arr) 完全不同

`if (arr)` 判断的是"arr 是否存在"(非 null/undefined),空数组 `[]` 是真值,恒进分支; `if (arr.length)` 判断"元素个数大于 0",空数组是 0(假)不进分支。两个意图都常见,写错就是线上 bug。同理 `if (obj.name)` 在 name 为 0 或 '' 时误判。

3.1.1 悬挂 else:C 的规则保留

与 C 相同,else 与最近的未配对 if 结合。生产规范建议任何 else 嵌套都加大括号,可读性优先。JS 没有语法问题,只有风格问题。

```

let x = 1, y = 2;
// 悬挂 else:else 属于内层 if(与 C 一致)
if (x === 1)
  if (y === 2) console.log('内层');
  else console.log('外层?不,还是内层的 else'); // 属于内层 if
// 上面代码等价于(不推荐,仅为解释规则):
if (x === 1) {
  if (y === 2) {
    console.log('内层');
  } else {
    console.log('还是内层的 else');
  }
}

```

§3.2 switch:严格 ===,没有隐式 fallthrough

对比项	C 的 switch	JS 的 switch
条件类型	整型/字符	任意值,用 === 严格比较
比较语义	按整数值	===(不转换类型)
fallthrough	默认穿透,漏 break 是经典 bug	默认不穿透(空 case 可共享)
default	可选,位置任意	同 C,习惯放最后
break 可省	不可省(除非故意穿透)	每 case 结尾 break 是惯例

```

function describe(color) {
  switch (color) {
    case 'red':
      return '红色';          // return 直接退出,不用 break
    case 'green':
      return '绿色';
    case 'blue':
      return '蓝色';
    default:
      return '未知颜色';
  }
}

console.log(describe('red'));    // 红色

// 共享 case:多个 case 执行同一段(与 C 的故意 fallthrough 相同)
function daysOf(month) {
  switch (month) {
    case 2:
      return 28;
    case 4:
    case 6:
    case 9:
    case 11:
      return 30;              // 4/6/9/11 共享
    default:
      return 31;
  }
}

console.log(daysOf(4));          // 30
console.log(daysOf(1));          // 31

// === 语义的坑:switch (x) 不转换类型
const n = '1';
switch (n) {
  case 1: console.log('数字 1'); break;    // 不会命中!n 是字符串
  case '1': console.log('字符串 1'); break; // 命中这里
  default: console.log('其他');
}

```

⚠ 常见误区 3.2:switch 忘了 break 会静默穿透吗?

JS 里忘写 break 确实会继续执行下一个 case 的语句(穿透),但每个 case 通常以 return/break 结束,且由于语言不强制,穿透在 JS 中比 C 更罕见。用 ESLint 的 no-fallthrough 规则可以静态拦截。生产建议:case 体内语句多时,用大括号包裹,声明局部变量时避免与穿透混淆。

工程建议 3.1:现代代码用"对象查找"替代 switch 长链

10 个 case 以上的 switch 建议改成对象查找(映射表),可读性更高、天然支持动态键:

```
const COLOR_CN = Object.freeze({
  red: '红色',
  green: '绿色',
  blue: '蓝色',
});
console.log(COLOR_CN['red'] ?? '未知颜色'); // 红色
```

§3.3 循环:四种形态与两种迭代

3.3.1 经典三循环:与 C 几乎相同

```
// for:与 C 完全一致
for (let i = 0; i < 5; i++) {
  process.stdout.write(i + ' ');
}
console.log(); // 0 1 2 3 4

// while
let j = 0;
while (j < 3) {
  process.stdout.write(j + ' ');
  j++;
}
console.log(); // 0 1 2

// do-while:至少执行一次(与 C 相同)
let k = 10;
do {
  console.log('执行一次:', k);
  k++;
} while (k < 3);
```

3.3.2 数组遍历的现代写法:for...of(ES2015)

```
const fruits = ['苹果', '香蕉', '橙子'];

// C 风格:下标遍历(JS 也支持)
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}

// 现代风格:for...of,直接取元素,无需下标
for (const f of fruits) {
  console.log(f);
}

// 需要下标时:用 entries()
for (const [i, f] of fruits.entries()) {
  console.log(`${i}: ${f}`);
}

// for...of 也能遍历字符串(按码元)与 Map/Set
for (const ch of 'abc') {
  console.log(ch); // a b c
}

const map = new Map([[ 'a', 1], [ 'b', 2]]);
for (const [key, value] of map) {
  console.log(key, value);
}
```

⚠ 常见误区 3.3:for...in 遍历数组的陷阱

for...in 遍历的是可枚举属性名(字符串),不是元素;它遍历数组得到的是 '0'、'1' 字符串下标,且会把数组上自定义的属性也遍历出来。生产规则:数组用 for...of,对象用 Object.keys/entries,for...in 基本不用(除非遍历普通对象的自身属性并接受原型链风险)。

```
const arr = ['a', 'b'];
arr.custom = 'extra';
for (const k in arr) {
  console.log(k); // 0 1 custom ← 把自定义属性也遍历了
}
for (const v of arr) {
  console.log(v); // a b ← 干净
}
```

§3.4 break / continue 与带标签跳转

C 的 break 只能跳出最近一层循环,JS 相同;但 JS 支持带标签的 break/continue,可以跳出任意多层——这是 C 的 goto 在现代代码里的安全替身(ESLint 规则也推荐标签而非 goto 风格跳转)。

```
// 双层循环找目标:跳出两层
outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (i === 1 && j === 1) {
      break outer;          // 直接跳出两层
    }
    console.log(`${i},${j}`);
  }
}
console.log('跳出两层');

// continue 标签:跳过外层循环的当前迭代
inner: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) {
      continue inner;        // 跳到外层下一轮,i 会 +1
    }
    console.log(`${i},${j}`);
  }
}

// continue 的常规用法:过滤
for (let i = 0; i < 10; i++) {
  if (i % 2 === 0) continue; // 跳过偶数
  process.stdout.write(i + ' ');
}
console.log(); // 1 3 5 7 9
```

§3.5 goto:JS 没有,但你有更好的工具

C 的 goto 用于:① 统一错误出口;② 跳出深层循环;③ 状态机跳转。JS 没有 goto 关键字,但三种场景都有现代替代:

C 的 goto 场景	JS 的替代
错误统一出口(err: cleanup; return)	try/finally(第9章)或提前 return
跳出深层循环	带标签 break / 提取成函数后 return
状态机跳转	while + switch 状态机模式,或状态对象


```
// 提取函数 + 提前 return,替代 goto 的"找出口"
function findIndex(matrix, target) {
  for (let i = 0; i < matrix.length; i++) {
    for (let j = 0; j < matrix[i].length; j++) {
      if (matrix[i][j] === target) {
        return [i, j];          // 直接返回,天然跳出两层
      }
    }
  }
  return [-1, -1];              // 未找到
}

const m = [[1, 2], [3, 4]];
console.log(findIndex(m, 3)); // [ 1, 0 ]

// 状态机:while + switch(替代 goto 跳转)
function smallStateMachine() {
  let state = 'start';
  while (state !== 'end') {
    switch (state) {
      case 'start':
        console.log('开始');
        state = 'work';
        break;
      case 'work':
        console.log('工作中');
        state = 'end';
        break;
    }
  }
}

smallStateMachine();
```

§3.6 循环变量捕获:var 的经典闭包 bug

这是 JS 面试与生产事故的"老朋友"。C 里 `for (int i = 0; ...)` 的 `i` 每轮都是独立的块变量;JS 的 `var` 是函数作用域,整个循环只有一个 `i`,循环结束后 `i` 保留最终值。于是回调函数(如 `setTimeout`)读取 `i` 时,读到的是同一个最终值。

```
// ❌ 经典 bug:全部输出 3
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log('var:', i), 10);
}
// 输出:3 3 3

// ✅ 修复一:用 let(每轮迭代独立绑定)
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log('let:', j), 10);
}
// 输出:0 1 2

// ✅ 修复二:立即执行函数捕获(旧代码风格)
for (var k = 0; k < 3; k++) {
  (function (n) {
    setTimeout(() => console.log('IIFE:', n), 10);
  })(k);
}
// 输出:0 1 2
```

★ 重点结论:let 的循环语义是"每轮新绑定"

for 循环里 `let i` 每次迭代都会创建一个新的绑定,回调捕获的是那一轮的 `i`——这是规范明确规定的语义(与 C 的块变量直觉一致)。生产铁律:循环变量一律 `let`,回调里捕获循环变量时检查是否为每轮值。异步回调细节见第9章事件循环。

§3.7 数组遍历三风格与无限循环对照

3.7.1 三风格总览:下标 / 迭代 / 函数式

```
const nums = [1, 2, 3, 4, 5];

// 风格一:for 下标(需要 index 或提前退出时)
for (let i = 0; i < nums.length; i++) {
  if (nums[i] === 3) break;
}

// 风格二:for...of(只读遍历的首选)
let sum = 0;
for (const n of nums) sum += n;
console.log(sum);           // 15

// 风格三:函数式(map/filter/reduce,第4章详讲)
const doubled = nums.map((n) => n * 2);
const evens = nums.filter((n) => n % 2 === 0);
const total = nums.reduce((acc, n) => acc + n, 0);
console.log(doubled, evens, total);

// forEach:副作用遍历
nums.forEach((n) => process.stdout.write(n + ' '));
console.log();

// 生产选型:需要 break/continue/await → for 系列;纯遍历 → for...of;
// 转换/过滤/聚合 → map/filter/reduce;其他副作用 → forEach
```

3.7.2 无限循环写法对照

```
// C 经典:for(;;) 或 while(1)
// JS 对应:
let running = true;
let steps = 0;
while (true) {
  steps++;
  if (steps >= 3) break;    // 必须要有退出条件!
}
console.log(steps);        // 3

// Node 服务器主循环常写成:事件循环驱动,不需要自己写死循环(见第9章)
```

3.7.3 遍历中修改数组:JS 没有"迭代器失效",但有新坑

C++ 在 vector 遍历中 erase 会使迭代器失效;JS 的数组没有这个问题,但 for...of 遍历中 push 会无限循环,删元素会跳过元素——**生产规则:遍历中不要增删原数组,先过滤出新数组再遍历。**

```
// 安全做法:过滤出新数组
const raw = [1, 2, 3, 4, 5, 6];
const kept = raw.filter((n) => n % 2 === 1); // 保留奇数
console.log(kept); // [ 1, 3, 5 ]
console.log(raw); // [ 1, 2, 3, 4, 5, 6 ] 原数组不变

// ❌ 错误示范:遍历中 splice
// for (let i = 0; i < raw.length; i++) {
//   if (raw[i] % 2 === 0) raw.splice(i, 1); // 下标错乱,跳过元素
// }
```

🚀 工程建议 3.2:循环里的大对象操作注意性能

循环体内避免:重复拼接字符串(用数组收集)、重复访问深层属性(先取局部变量)、每次迭代创建闭包(提出来)。10 万级循环里, `const t = obj.a.b` 提出来能省一半时间;字符串收集后 `join` 比 `+=` 快一个数量级。用 Node 的 `--prof` 或浏览器 Performance 面板定位热点后再优化,不要盲目微优化。

本章速查表

主题	速查要点
if 条件	按真值:仅 6 个假值(false 0 -0 " null undefined NaN)
if (arr)	判断"数组存在";if (arr.length) 判断"有元素",别混
悬挂 else	与 C 相同;else 归最近的 if;生产全加大括号
switch 比较	用 ===,不转类型:'1' 不命中 case 1
switch fallthrough	默认不穿透;case 4: case 5: 共享合法
for	与 C 相同;循环变量用 let
do-while	至少执行一次,与 C 相同
for...of	数组/字符串/Map/Set 的值遍历,首选
for...in	遍历可枚举属性名;数组别用;生产基本不用
break/continue 标签	outer: for(...) 后 break outer;替代 goto 跳层
goto	没有;提前 return / 标签 break / 状态机替代
var 闭包 bug	var 是函数作用域,回调全读到最终值;let 每轮新绑定
遍历风格	转换 map;筛选 filter;聚合 reduce;副作用 forEach;要 break 用 for
遍历中改数组	禁止;先 filter 出新数组
无限循环	while(true) + break;事件循环由运行时驱动

最小必会清单

- 能列出 6 个假值,并说出 if (count) 与 if (count > 0) 的语义差异
- 能写出 switch 的 === 语义案例,并解释 '1' 不命中 case 1

- 能写出共享 case 的 switch(4/6/9/11 月共享 30 天)
- 能说出 for...of 与 for...in 的区别与各自适用场景
- 能写出带标签的 break 跳出双层循环
- 能说出 JS 没有 goto 后三种替代方案
- 能复现 var 闭包 bug 并用 let 修复,并解释原因
- 能写出 for...of、entries()、forEach、map/filter/reduce 五种遍历
- 能解释为什么遍历中 splice 会跳过元素,并给出安全写法
- 能用 while(true) + break 写出带退出条件的循环

自测题

Q 1. 写出输出:`const a = []; if (a) console.log('A'); if (a.length) console.log('B')`

✓ 参考答案

只输出 A。[] 是真值(对象都是真值),a.length 是 0(假值)。

Q 2. 写出输出:`const x = '2'; switch (x) { case 2: console.log('数字'); break; case '2': console.log('字符串'); break; default: console.log('其他'); }`

✓ 参考答案

输出"字符串"。switch 用 ===,字符串 '2' 不等于数字 2。

Q 3. 下面的代码输出什么?为什么?`for (var i = 0; i < 3; i++) { setTimeout(() => console.log(i), 0); }`

✓ 参考答案

输出 3 3 3。var 是函数作用域,只有一个 i,循环结束后 i=3,回调执行时全部读到 3。改成 let 输出 0 1 2。

Q 4. 如何用带标签的 break 从双层循环中跳出?请给出代码。

✓ 参考答案

`outer: for (...) { for (...) { if (条件) break outer; } },break outer 直接结束外层循环。`

Q 5. for...of 遍历数组 arr 时,arr.custom = 1 会被遍历到吗?for...in 呢?

✓ 参考答案

for...of 不会(只取元素);for...in 会(遍历所有可枚举字符串属性名,包括 custom)。

Q 6. 写一个 do-while 循环,打印 1 到 3,并说明它和 while 的区别。

✓ 参考答案

`let i = 1; do { console.log(i); i++; } while (i <= 3);`。do-while 先执行循环体再判断,至少执行一次。

章末实验题:文本菜单与素数筛选统计器

实验 3:文本菜单与素数筛选统计器

实验目标:实现一个命令行交互工具:菜单驱动(1 筛选素数 / 2 统计数组 / 3 矩阵查找 / 0 退出),强制覆盖全章流程控制知识点。

实验要求:

- 任务 1(覆盖:if/else、真值判断):菜单输入用真值规则防御空输入,选项用 if 链分发
- 任务 2(覆盖:switch、共享 case):选项处理改用 switch,支持 '1'/'2'/'3' 数字与 'q' 共享退出路径
- 任务 3(覆盖:for、while、break/continue):素数筛选用双重 for + continue;读取输入用 while 循环
- 任务 4(覆盖:do-while):统计子菜单用 do-while 保证至少执行一次
- 任务 5(覆盖:带标签 break、for...of、entries):矩阵查找用标签 break 提前退出;遍历统计用 for...of 与 entries
- 任务 6(覆盖:var 闭包 bug 与 let):在工具内演示 setTimeout 捕获循环变量时 var 与 let 的差异
- 任务 7(覆盖:对象查找替代 switch 长链):把等级映射表用对象查找实现

实验环境:Node.js 20+,交互式运行 `node menu.js`;或把输入重定向为文件。

第一步 写纯函数:isPrime(n)、filterPrimes(list)、matrixFind(matrix, target)、statList(list),全部与 UI 解耦

第二步 写菜单循环:while(true) + switch(choice),'q' 退出;输入用 readline 的 question 封装成 await

第三步 后写演示区:展示标签 break、var/let 闭包差异、对象查找

✅ 参考答案要点

核心:纯函数(素数/统计/查找)+ while(true) 菜单循环 + switch 分发 + readline 异步输入。约 120 行,覆盖全章全部知识点。

```
// menu.js — Node.js 20+ 运行:node menu.js
'use strict';
const readline = require('node:readline/promises');
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

// 1. 素数判断:for + continue(覆盖:for、continue、真值)
function isPrime(n) {
  if (n < 2) return false;
  if (n === 2) return true;
  if (n % 2 === 0) return false;
  for (let d = 3; d * d <= n; d += 2) {
    if (n % d === 0) return false;
  }
  return true;
}

// 2. 筛选素数:for...of + 数组 push(覆盖:for...of、数组)
function filterPrimes(list) {
  const out = [];
  for (const n of list) {
    if (isPrime(n)) out.push(n);
  }
  return out;
}

// 3. 矩阵查找:带标签 break(覆盖:标签、break)
function matrixFind(matrix, target) {
  outer: for (let i = 0; i < matrix.length; i++) {
    for (let j = 0; j < matrix[i].length; j++) {
      if (matrix[i][j] === target) {
        return [i, j];
      }
    }
  }
  return [-1, -1];
}

// 4. 统计:do-while + while(覆盖:do-while、while、break)
function statList(list) {
  let sum = 0;
  let max = list.length > 0 ? list[0] : 0;
  let count = 0;
  let i = 0;
  if (list.length > 0) {
    do {
      const v = list[i];
      sum += v;
      if (v > max) max = v;
      count++;
      i++;
    } while (i < list.length);
  }
}
```



```
    return { sum, max, count, avg: count === 0 ? 0 : Math.round((sum / count) * 100) / 100 };
}
```

// 5. 对象查找替代 switch 长链(覆盖:对象映射、??、真值)

```
const GradeCN = Object.freeze({
  A: '优秀',
  B: '良好',
  C: '及格',
  D: '不及格',
});
function gradeOf(avg) {
  if (avg >= 90) return GradeCN.A ?? '优秀';
  if (avg >= 75) return GradeCN.B;
  if (avg >= 60) return GradeCN.C;
  return GradeCN.D;
}
```

// 6. 演示 var 闭包 bug 与 let 修复(覆盖:闭包、let/var)

```
function closureDemo() {
  const results = [];
  for (var v = 0; v < 3; v++) {
    results.push(() => v); // 三个函数共享同一个 v
  }
  for (let l = 0; l < 3; l++) {
    results.push(() => l); // 每轮独立绑定
  }
  const nums = results.map((f) => f());
  console.log('var 段全为 3,let 段 0..2:', nums.join(','));
}
```

// 7. 菜单主循环:while + switch(覆盖:switch、break、while、真值)

```
async function main() {
  const data = [10, 23, 47, 51, 97, 4, 13];
  const matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
  console.log('==== 流程控制实验菜单 =====');
  while (true) {
    const choice = (await rl.question(
      '[1]素数筛选 [2]数组统计 [3]矩阵查找 [4]闭包演示 [q]退出\n> '
    )).trim().toLowerCase();
    switch (choice) {
      case '1': {
        console.log('素数:', filterPrimes(data).join(' '));
        break;
      }
      case '2': {
        const st = statList(data);
        console.log(`和=${st.sum} 最大=${st.max} 个数=${st.count} 平均=${st.avg}`);
        console.log(`平均等级:${gradeOf(st.avg)}`);
        break;
      }
      case '3': {
        const pos = matrixFind(matrix, 6);
        if (pos[0] === -1) {
          console.log('未找到');
        } else {

```

```

        console.log(`找到 6 在 (${pos[0]},${pos[1]})`);
    }
    break;
}
case '4':
    closureDemo();
    break;
case 'q':          // 共享退出路径
case 'Q':
    console.log('再见');
    rl.close();
    return;
default:
    console.log('未知选项,重新输入');
}
}
}

main().catch((err) => {
    console.error('程序异常:', err);
    rl.close();
});

```

设计说明:纯函数与 UI 分离便于测试;菜单用 while(true) + switch(choice) 结构,case 内大括号包裹声明局部变量避免穿透歧义;输入 trim().toLowerCase() 后用真值检查防御空输入;闭包演示把 var/let 差异可视化;gradeOf 用对象查找 + ?? 默认值。删除注释中的 for (var v ...) 用 let 即可观察修复效果。

下一章预告:第4章 函数与方法 —— 函数是一等公民、this 绑定四规则、闭包与高阶函数,理解这些才能真正读懂 JS 代码。